

UNIT-I

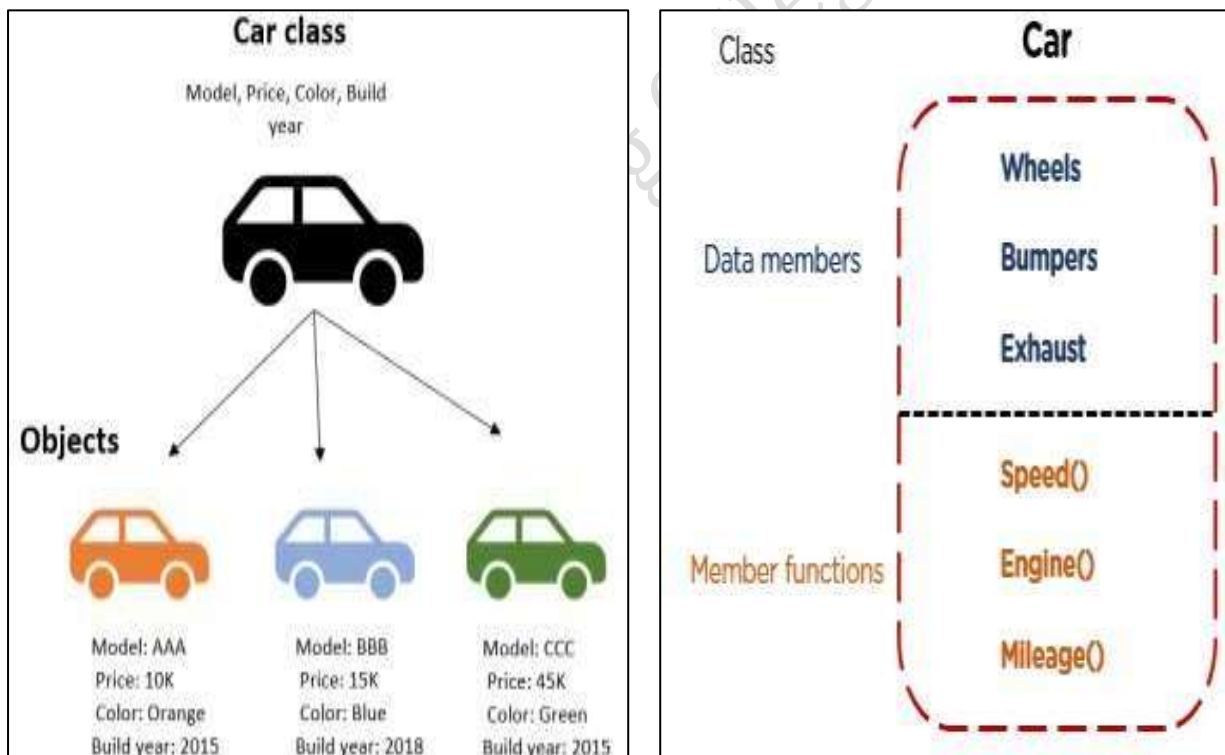
1. Introduction

- C++ is an object oriented programming language & is an extension of c.
- C++ is an object-oriented programming language developed by Bjarne Stroustrup starting in 1979 at Bell Labs.
- He called the language C with classes.

2. Elements/Features of Object Oriented programming

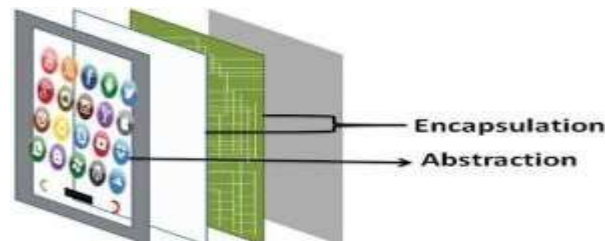
2.1. Classes & Objects

- An object is a basic unit in object-oriented programming. An object contains data and methods or functions that operate on that data. Objects take up space in memory.
- A class, on the other hand, is a blueprint of the object. Conversely, an object can be defined as an instance of a class. A class contains a skeleton of the object and does not take any space in the memory.



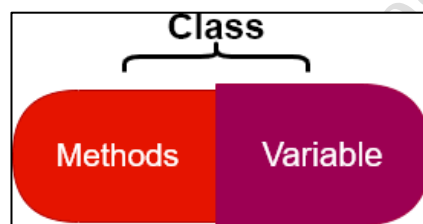
2.2. Abstraction

- Abstraction is the process of hiding irrelevant information from the user.
- Using abstraction in programming, we can hide unnecessary details from the user. By using abstraction in our application, the end user is not affected even if we change the internal implementation.



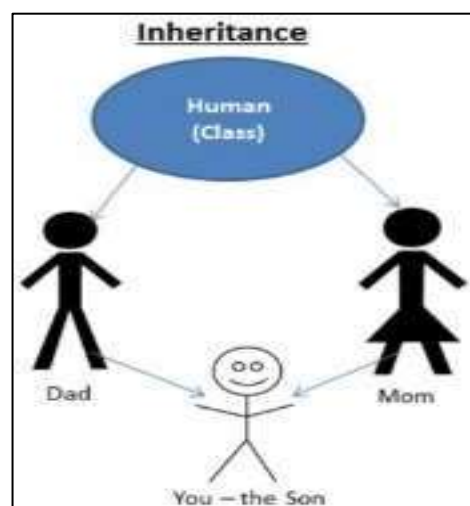
2.3. Encapsulation

- Encapsulation is the process using which data and the methods or functions operating on them are bundled together.
- By doing this, data is not easily accessible to the outside world. In OOP we achieve encapsulation by making data members as private and having public functions to access these data members.



2.4. Inheritance

- The ability to inherit the properties of one class to another, or inherit the properties from a base class to an inherited class is known as the concept of Inheritance.
- A class that is inherited by other classes is termed as super class or parent class or base class, whereas a class that extends another class is termed as sub-class or child class.



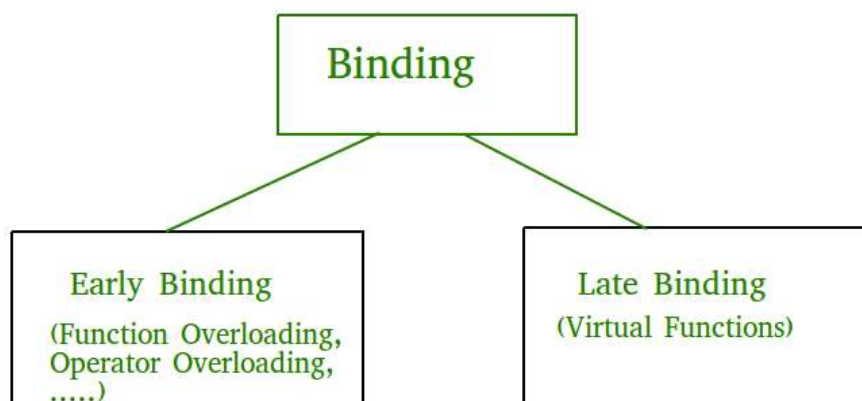
2.5. Polymorphism

- Polymorphism means many forms.
- Polymorphism is an important feature of OOP and is usually implemented as operator overloading or functionoverloading. Operator overloading is a process in which an operator behaves differently in different situations. Similarly, in function overloading, the same function behaves differently in different situations.



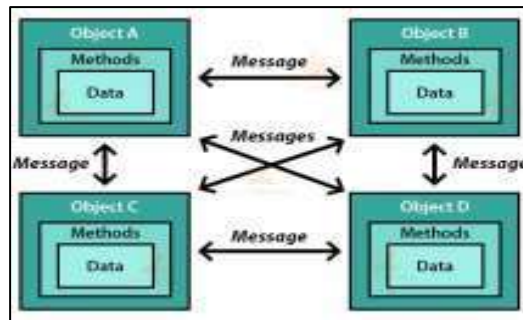
2.6. Dynamic Binding

- OOP supports dynamic binding in which function call is resolved at runtime.
- This means that the code to be executed as a result of a function call is decided at runtime. Virtual functions are an example of dynamic binding.



2.7. Message Passing

- In OOP, objects communicate with each other using messages. When objects communicate, information is passed back and forth between the objects.
- A message generally consists of the object name, method name and actual data that is to be sent to another object.



3. Advantages of OOP

- OOPS can be comfortably upgraded.
- Using inheritance redundant program codes can be eliminated & use of previously defined classes may be continued.
- OOP languages have standard class library.

4. Uses of OOP

- Object-Oriented DBMS
- Office automation software
- AI and expert systems
- CAD/CAM software
- Network programming
- System software

5. Specifying/Defining a Class

- A class is defined in C++ using keyword `class` followed by the name of class.
- The body of class is defined inside the curly brackets and terminated by a semicolon at the end.

Syntax:

keyword user-defined name

```

class ClassName
{
    Access specifier:           //can be private,public or protected
    Data members;              // Variables to be used
    Member Functions() { }    //Methods to access data members
};                               // Class name ends with a semicolon
  
```

Example:

```
class Student
{
    public:
    int id;           //field or data member
    int fees;         //field or data member
    String name;      //field or data member
    void studying() { }
};
```

- Here, we defined a class named student.
- The variables id, fees & name declared inside the class are known as data members. And, the functions studying() is known as member functions of a class.

6. Creating Objects

- When a class is defined, only the specification for the object is defined; no memory or storage is allocated.
- To use the data and access functions defined in the class, you need to create objects.

Syntax:

```
ClassName ObjectName;
```

Example:

```
Student s1,s2,s3;
```

Here, s1,s2,s3 are objects of the students class

7. Accessing data members

We can access the data members of a class by using a . (dot) operator.

Syntax:

```
ObjectName. DataMember;
```

Example:

```
S1.id;
```

```
S1.name;
```

8. Accessing Member Function

We can access the member functions of a class by using a . (dot) operator.

Syntax:

```
ObjectName.MemberFunction();
```

Example:

```
S1.studying();
```

9. Class and object Program

Let's see an example of class that has two fields: id and name. It creates instance of the class, initializes the object and prints the object value.

Program:

```
#include <iostream>
using namespace std;
class Student{
    public:
    int roll;//data member (also instance variable)
    string name;//data member(also instance variable)
};

int main() {
    Student s1; //creating an object of Student
    s1.roll = 201;
    s1.name = "Neha Ray";
    cout<<s1.roll<<endl;
    cout<<s1.name<<endl;
    return 0;
}
```

Output:

```
201
Neha Ray
```

9.1. C++ Class Example: Initialize and Display data through method

Program:

```
#include <iostream>
using namespace std;
class Student {
    public:

    int id;           //data member (also instance variable)
    string name;      //data member(also instance variable)

    void insert(int i, string n){
        id = i;
        name = n;
    }

    void display(){
        cout<<id<<" "<<name<<endl;
    }
};

int main() {
    Student s1; //creating an object of Student
    Student s2; //creating an object of Student
    s1.insert(201, "Sonoo");
    s2.insert(202, "Nakul");
    s1.display();
    s2.display();
    return 0;
}
```

Output:

```
201 Sonoo
202 Nakul
```

9.2. C++ Class Example: Store and Display Employee Information

Let's see another example of C++ class where we are storing and displaying employee information using method.

Program:

```
#include <iostream>
using namespace std;
class Employee {
public:

    int id;//data member (also instance variable)
    string name;//data member(also instance variable)
    float salary;

    void insert(int i, string n, float s){
        id = i;
        name = n;
        salary = s;
    }

    void display(){
        cout<<id<<" " <<name<<" " <<salary<<endl;
    }
};

int main(void) {
    Employee e1; //creating an object of Employee
    Employee e2; //creating an object of Employee
    e1.insert(201, "Sonoo", 990000);
    e2.insert(202, "Nakul", 29000);
    e1.display();
    e2.display();
    return 0;
}
```

Output:

```
201 Sonoo 990000
202 Nakul 29000
```


10. Defining Member Function

Member functions can be defined at two places inside the class and outside class.

1. Defining function inside the class
2. Defining function outside the class

10.1. Inside Class Definition

Program:

```
#include <iostream>
using namespace std;
class employee{
public:
void working(){                               //definition within the class
    cout<<"employee is working";
}
};
int main(){
    employee e;
    e.working();
    return 0;
}
```

Output:

Employee is working

10.2. Defining function outside of the class

- Scope resolution operator :: along with class name is used to define a function outside of class.
- Member function must be declared inside the class.

Syntax:

```
function_return_type class_name::function_name(parameters){}
```

Program:

```
#include <iostream>
using namespace std;
class employee{
public:
void working();                               //function declaration
};
void employee::working(){                     //definition outside the class
    cout<<"employee is working";
}
int main(){
    employee e;
    e.working();
    return 0;
}
```

Output:

Employee is working

11. Static data member and Member Function

11.1. Static data member

- We can define class members static using **static** keyword.
- When we declare a member of a class as static it means no matter how many objects of the class are created, there is only one copy of the static member.
- A static member is shared by all objects of the class.
- We cannot initialize static data members within the class, it can be initialized outside the class using scope resolution operator to identify which class it belongs to.

Program 1:

```
#include <iostream>
using namespace std;
class student{
public:
static string course="BCA"; //error
};

int main(){
    student s;
    cout<<s.course;
}
```

Output:

Above program will throw error, we cannot initialize static members inside the class

Program 2:

```
#include <iostream>
using namespace std;
class student{
public:
static string course;
};

string student::course="BCA";    //Initialization of static member

int main(){
    student s;
    cout<<s.course;
}
```

Output:

BCA

11.2. Static Member Function

- Static member functions are defined using static keyword.
- A static member function can be called even if no objects of the class exist and the static functions are accessed using only the class name and the scope resolution operator ::
- A static member function can only access static data member.
- Static members can be called using object and class name but non static members can only be called using object, but not by using class name.

Program:

```
#include <iostream>
using namespace std;
class student{
public:
static void demo(){
    cout<<"i am static function";
}
};

int main(){
    student::demo(); //calling static function
}
```

Output:

i am static function

12. Access Specifiers/Modifiers:

- Access Specifiers are basically used to implement Data Hiding. Data Hiding in C++ Refers to restricting access to data members of a class from outside the class.
- This is required when we don't want other functions and classes to tamper with our class data. However, it is also important to make some member functions accessible so that the hidden data can be manipulated indirectly.
- This is where Access Specifiers come into the picture. It allows us to determine which class members are accessible to other classes and functions, and which class members are restricted from access.
- By default all the members of class are private.

12.1. Types of C++ Access Modifiers

In C++, there are 3 access modifiers:

1. public
2. private
3. protected

12.1.1 public

- The public keyword is used to create public members (data and functions).
- The public members are accessible from any part of the program.

Program:

```
#include <iostream>
using namespace std;

// define a class
class Sample {

// public elements
public:
    int age;

    void displayAge() {
        cout << "Age = " << age << endl;
    }
};

int main() {
    Sample obj1; // declare a class object
    cout << "Enter your age: ";
    cin >> obj1.age; // store input in age of the obj1 object
    obj1.displayAge(); // call class function
    return 0;
}
```

Output:

Enter your age: 12
Age = 12

Explanation:

- In this program, we have created a class named Sample, which contains a public variable age and a public function displayAge().
- In main(), we have created an object of the Sample class named obj1. We then access the public elements directly by using the codes obj1.age and obj1. displayAge().

12.1.2 Private

- The private keyword is used to create private members (data and functions).
- The private members can only be accessed from within the class.
- However, friend classes and friend functions can access private members.

Program 1:

```
#include <iostream>
using namespace std;
class Student
{
    private:
        int id;           //data member (also instance variable)
        string name;      //data member(also instance variable)

    public:
        void insert(int i, string n){
            id = i;
            name = n;
        }

        void display(){
            cout<<id<<" "<<name<<endl;
        }
};

int main() {
    Student s1;           //creating an object of Student
    s1.id=101;             //ERROR
    s1.name="NISHA";      //ERROR
    s1.display();
    return 0;
}
```

Explanation:

- Above program will throw error because, we are accessing private data members id and name outside of the class.

Program 2:

```
#include <iostream>
using namespace std;
class Student
{
    private:
        int id;           //data member (also instance variable)
        string name;      //data member(also instance variable)

    public:
        void insert(int i, string n){
            id = i;
            name = n;
        }

        void display(){
            cout<<id<<" "<<name<<endl;
        }
};

int main(){
    Student s1;           //creating an object of Student
    s1.insert(101,"NISHA");
    s1.display();
    return 0;
}
```

Output:

101NISHA

12.1.3 Protected

- Protected access specifier is used in inheritance.
- The protected keyword is used to create protected members (data and function).
- Protected members cannot be accessed from outside the class, however, they can be accessed in inherited classes.

Program:

```
#include <iostream>
using namespace std;

// base class
class Parent
{
    // protected data members
    protected:
    int id_protected;
};

// sub class or derived class from public base class
class Child : public Parent
{
    public:
    void setId(int id)
    {
        // Child class is able to access the inherited
        // protected data members of base class
        id_protected = id;
    }

    void displayId(){
        cout << "id_protected is: " << id_protected << endl;
    }
};

int main() {
    Child obj1;
    obj1.setId(81);
    obj1.displayId();
    return 0;
}
```

Output:

id_protected is: 81

UNIT-II

1. Constructor

- **Constructor in C++** is a special method that is invoked automatically at the time of object creation.
- It is used to initialize the data members of new objects generally. The constructor in C++ has the same name as the class or structure.
- Constructor is invoked at the time of object creation. It constructs the values i.e. provides data for the object which is why it is known as constructors.

1.1 Characteristics of Constructors

- Constructor is a member function of a class, whose name is same as the class name.
- Constructor is a special type of member function that is used to initialize the data members for an object of a class automatically, when an object of the same class is created.
- Constructor is invoked at the time of object creation. It constructs the values i.e. provides data for the object that is why it is known as constructor.
- Constructor do not return value, hence they do not have a return type.

1.2 Defining a Constructor

Constructor can be defined inside the class declaration or outside the class declaration.

1. Syntax for defining the constructor within the class

```
class-name(){  
    //constructor definition  
}
```

Example:

```
class student{  
    public:  
    student(){  
    }  
};
```

2. Syntax for defining the constructor outside the class

```
class-name :: class-name() {  
    //constructor definition  
}
```

Example:

```
class student{  
    public:  
    student();  
};  
student::student(){}
```


1.3 Types of constructor

- 1.3.1 Default Constructor
- 1.3.2 Parameterized Constructor
- 1.3.3 Copy Constructor

1.3.1 Default Constructor

A constructor which has no argument is known as default constructor. It is invoked at the time of creating object.

Syntax:

```
className (){ }
```

Program:

```
#include <iostream>
using namespace std;
class Employee{
    public:
        Employee(){
            cout<<"Default Constructor Invoked"<<endl;
        }
};

int main(){
    Employee e1;           //creating an object of Employee
    Employee e2;
    return 0;
}
```

Output:

```
Default Constructor Invoked
Default Constructor Invoked
```

1.3.2 Parameterized Constructor:

A constructor which has parameters is called parameterized constructor. It is used to provide different values to distinct objects.

Syntax:

```
className (parameter List){ }
```

Program:

```
#include <iostream>
using namespace std;
class Employee {
public:
    int id;                //data member (also instance variable)
    string name;           //data member(also instance variable)
    float salary;

    Employee(int i, string n, float s){
        id = i;
        name = n;
        salary = s;
    }

    void display(){
        cout<<id<<" " <<name<<" " <<salary<<endl;
    }

};
int main(){
    Employee e1 =Employee(101, "Sonoo", 890000); //creating an object of Employee
    Employee e2=Employee(102, "Nakul", 59000);
    e1.display();
    e2.display();
    return 0;
}
```

Output:

```
101 Sonoo 890000
102 Nakul 59000
```

1.3.3 Copy Constructor

- A **copy constructor** is a member function that initializes an object using another object of the same class. In simple terms, a constructor which creates an object by initializing it with an object of the same class, which has been created previously is known as a **copy constructor**.
- Copy constructor is used to initialize the members of a newly created object by copying the members of an already existing object.
- Copy constructor takes a reference to an object of the same class as an argument.

Syntax:

```
className (className &object){ }
```

Example:

```
class student{
    Public:
    Student(student &s1){           //copy constructor
    }
}
```

Calling Copy Constructor:

Copy constructors can be called in 2 ways

Syntax:

```
className obj1 (obj2);
OR
className obj1=obj2;
```

Program:

```
#include <iostream>
using namespace std;
class A{
    public:
    int x;

    A(int a){           // parameterized constructor.
        x=a;
    }

    A(A &i) {           // copy constructor
        x = i. x;
    }

};
```

```
int main(){  
    A a1(20);    // Calling the parameterized constructor.  
    A a2(a1);    // Calling the copy constructor.  
    cout<<a2.x;  
    return 0;  
}
```

Output:

20

Explanation:

The line A a2 (a1) will call copy constructor and assign the value of object a1 into a2.

Note: Defining body of copy constructor is not required, compiler supplies its own copy constructor if not provided by user.

1.4 Constructor can be defined outside the class:

- The constructor can be defined outside the class but it has to be declared inside the class. Here, you will use the scope resolution operator.

Program:

```
#include <iostream>  
using namespace std;  
class Employee {  
    public:  
    Employee();    //constructor declaration  
};  
Employee::Employee(){    //constructor definition outside of class  
    cout<<"Default Constructor Invoked"<<endl;  
}  
  
int main(){  
    Employee e1;    //creating an object of Employee  
    Employee e2;  
    return 0;  
}
```

Output:

Default Constructor Invoked
Default Constructor Invoked

1.5 Constructor overloading

- Sometimes there is a need of initializing an object in different ways. This can be done using constructor overloading.
- Constructor overloading is a concept in which one class can have multiple constructors with different parameters.
- Overloaded constructors essentially have the same name (exact name of the class) and different by number and type of arguments.
- A constructor is called depending upon the number and type of arguments passed.
- While creating the object, arguments must be passed to let compiler know, which constructor needs to be called.

Program:

```
#include <iostream>
using namespace std;
class Person {
    private:
        int age;
    public:

    Person(){                                // 1. Constructor with no arguments
        age = 20;
    }

    Person(int a){                            // 2. Constructor with an argument
        age = a;
    }

    int getAge()
    return age;
}

};

int main() {
    Person person1, person2(45);
    cout << "Person1 Age = " << person1.getAge() << endl;
    cout << "Person2 Age = " << person2.getAge() << endl;
    return 0;
}
```

Output:

Person1 Age = 20
Person2 Age = 45

Exaplanation:

In this program, we have created a class Person that has a single variable age.

We have also defined two constructors Person() and Person(int a):

When the object person1 is created, the first constructor is called because we have not passed any argument. This constructor initializes age to 20.

When person2 is created, the second constructor is called since we have passed 45 as an argument. This constructor initializes age to 45.

1.6 Constructor with Default Argument

- Default arguments of the constructor are those which are provided in the constructor declaration. If the values are not provided when calling the constructor the constructor uses the default arguments automatically.

1.6.1 Characteristics for defining the default arguments:

Following are the rules of declaring default arguments -

- The values passed in the default arguments are not constant. These values can be overwritten if the value is passed to the function. If not, the previously declared value retains.
- During the calling of function, the values are copied from left to right.
- All the values that will be given default value will be on the right.

Program:

```
#include<iostream>
using namespace std;
class Simple{
    int d1,d2,d3;
    public:
        Simple(int a, int b=9, int c=8) {
            d1 = a;
            d2 = b;
            d3 = c;
        }
        void printData();
};
void Simple :: printData(){
    cout<<"The value of data1, data2 and data3 is"<<d1<<" "<<d2<<"and"<< d3<<endl;
}

int main(){
    Simple s(12, 13);
    s.printData();
    return 0;
}
```

Output:

The value of data1, data2 and data3 is 12, 13 and 8

Explanation

- The values “12” and “13” are passed to the variables “a” and “b” but the value for the variable “c” was not passed that’s why constructors set the value “8” which was the default value for the variable “c”.

2 Destructor

- Destructor is also a special member function like constructor. Destructor destroys the class objects created by constructor.
- Destructor has the same name as their class name preceded by a tilde (~) symbol.
- It is not possible to define more than one destructor.
- The destructor is only one way to destroy the object create by constructor. Hence destructor can-not be overloaded.
- Destructor neither requires any argument nor returns any value.
- It is automatically called when object goes out of scope.
- Destructor release memory space occupied by the objects created by constructor.

Syntax for defining the destructor

```
~class-name(){  
    //destructor definition  
}
```

Example:

```
class student{  
    public:  
    ~student(){  
    }  
};
```

Program:

```
#include<iostream>
using namespace std;
class Test
{
    public:
        Test(){
            cout<<"\n Constructor executed";
        }

        ~Test(){
            cout<<"\n Destructor executed";
        }
};
int main()
{
    Test t,t1,t2,t3;
    return 0;
}
```

Output:

Constructor executed
Constructor executed
Constructor executed
Constructor executed
Destructor executed
Destructor executed
Destructor executed
Destructor executed

2.3 Properties of Destructor:

- Destructor function is automatically invoked when the objects are destroyed.
- It cannot be declared static or const.
- The destructor does not have arguments.
- It has no return type not even void.
- An object of a class with a Destructor cannot become a member of the union.
- A destructor should be declared in the public section of the class.

3 Constructor and destructor calling order

- The objects which are created first will call the constructor first.
- The objects which are created at the end will be destroyed first and calls the destructor first.

Program:

```
#include<iostream>
using namespace std;
class student
{
    int id;
    public:

    student(int i){
        id=i;
        cout<<"constructor called "<<id<<endl;
    }

    ~student(){
        cout<<"destructor called "<<id<<endl;
    }

};

int main(){
    student s1(10),s2(20),s3(30);
    return 0;
}
```

Output:

```
constructor called 10
constructor called 20
constructor called 30
destructor called 30
destructor called 20
destructor called 10
```

Explanation

- In the above program 3 objects are created s1, s2 and s3.
- First s1 will call the constructor and will be initialized with 10
- Then s2 will call the constructor and will be initialized with 20
- At the end s3 will call the constructor and will be initialized with 30
- After constructor call, now s3 object will call the destructor first, then s2 and at the end s1 object will call the destructor.

4 Operator Overloading

- Operator Overloading is a mechanism which provides an additional meaning to c++ operator.
- It provides you the opportunity to redefine the c++ operator to behave with user-defined data-type in the same way as they behave with built-in data types.
- For example, we can overload an operator '+' in a class like String so that we can concatenate two strings by just using +.

Syntax to create a member operator function:

To overload an operator, a special operator function is defined inside the class as:

```
class className
{
    ... ..
    Public:
        returnType operator symbol (arguments)
        {
            ... ..
        }
    ... ..
};
```

- Here, returnType is the return type of the function.
- The returnType of the function is followed by operator keyword.
- Symbol is the operator symbol you want to overload. Like: +, <, -, ++
- You can pass arguments to the operator function in similar way as functions.

Syntax to call operator function:

```
Operator_nameObject_name;
```

Example:

```
+obj;
```

Program to overload '+' operator:

```
#include <iostream>
using namespace std;
class Test{
    private:

        int count;
        string fname,lname,result;
    public:

        Test(){
            count =12;
            fname="sahyog";
            lname="college";
        }

        void operator +(){
            count = count+5;
            result=fname+" "+lname;
        }

        void Display() {
            cout<<"Count: "<<count;
            cout<<"\nResult: "<<result;
        }
};

int main(){
    Test t;
    +t;
    t.Display();
    return 0;
}
```

//definition of '+' function

//this calls "function void operator +()" function

Output:

Count: 17
Result: Sahyog College

Explanation:

Note:In the above program whenever "+" function will be called then value of count variable will be added by 5 and string fname & lname will also get concatenated (added) for 't' object.

Program to overload pre decrement "--" operator:

```

#include <iostream>
using namespace std;
class student
{
    public:
        string name;
        int age;

        student(string n,int a){
            name=n;
            age=a;
        }

        void operator --(){
            --age;
        }

};

int main(){
    student s("neha",20);
    cout<<"before calling -- operator function:\n";
    cout<<"name: "<<s.name<<"\n age: "<<s.age;
    --s; //calling " - -"function
    cout<<"\nAfter calling -- operator function-1st time:\n";
    cout<<"name: "<<s.name<<"\n age: "<<s.age;
    --s;
    cout<<"\nAfter calling -- operator function-2nd time\n";
    cout<<"name: "<<s.name<<"\n age: "<<s.age;

    return 0;
}

```

Output:

```

before calling -- operator function:
name: neha
age: 20
After calling -- operator function-1st time:
name: neha
age: 19
After calling -- operator function-2nd time
name: neha
age: 18

```

Note: in the above program we have overloaded pre-decrement operator(--). Whenever "--()" function will be called, then age will be decremented by 1.

Program to Overload post-decrement (--) Operator:

The operator symbol for both **pre-decrement(--i)** and **post-decrement(i--)** are the same. Hence, we need two different function definitions to distinguish between them. This is achieved by passing a dummy **int** parameter in the post-decrement.

Program:

```
#include <iostream>
using namespace std;
class student{
    public:
        string name;
        int age;

        student(string n,int a){
            name=n;
            age=a;
        }

        void operator --(int){ //definition of post-decrement '--'function
            age--;
        }

};

int main(){
    student s("neha",20);
    cout<<"before calling -- operator function:\n";
    cout<<"name: "<<s.name<<"\n age: "<<s.age;
    s--;
    cout<<"\nAfter calling -- operator function-1st time:\n";
    cout<<"name: "<<s.name<<"\n age: "<<s.age;
    s--;
    cout<<"\nAfter calling -- operator function-2nd time\n";
    cout<<"name: "<<s.name<<"\n age: "<<s.age;
    return 0;
}
```

Output:

```
before calling -- operator function:
name: neha
age: 20
After calling -- operator function-1st time:
name: neha
age: 19
After calling -- operator function-2nd time
name: neha
age: 18
```

Note:

- There is no difference between post-decrement & pre-decrement.
- But function definition of both the operators are quite different, to distinguish between the definition pre-decrement(`--()`) and post-decrement(`--()`) a dummy "int" parameter is passed as parameter in the definition of Post-decrement.
- Eg: void operator `--()` //definition of pre decrement
- Eg: void operator `--(int)` //definition of post-decrement

Program to Overload pre-increment "++" operator:

```
#include<iostream>
using namespace std;
class bank
{
    public:
    int amount,total=0;

    bank(int a){
        amount=a;
    }

    void operator ++(){
        amount++;
    }
};

int main(){
    bank b(100);
    cout<<"amount before calling ++(): "<<b.amount;
    ++b;
    cout<<"\namount after calling ++(): "<<b.amount;
    return 0;
}
```

Output:

```
amount before calling ++(): 100
amount after calling ++(): 101
```

Add Two objects using overloaded operator function:

```
#include<iostream>
using namespace std;
class addition{

    private:

    int number,n2,add;

    public:

    addition(int x) {
        number=x;
    }

    void operator +(addition x1){
        cout<<"First number:"<<number;
        cout<<"\nSecond Number:"<<x1.number;
        add= number + x1.number;
        cout<<"\nAddition="<<add;
    }
};

int main(){
    addition num1(10);
    addition num2(20);
    num1+num2;
    return 0;
}
```

here we adding 2 objects,this statement means,num1 is calling +() function and num2 is passed as a parameter

Output:

First number:10 Second Number:20 Addition=30

Program : 2nd way to call operator function:

```
#include<iostream>
using namespace std;
class addition{
private:
int number,n2,add; public:

    addition(int x){
        number=x;
    }

    void operator +(addition x1){

        cout<<"First number:"<<number;
        cout<<"\nSecond Number:"<<x1.number;
        add=this->number + x1.number;
        cout<<"\nAddition="<<add;
    }

};

int main(){
    addition num1(10);
    addition num2(20);
    num1.operator +(num2); //2nd way of calling operator +() function
    return 0;
}
```

Output:

First number:10
Second Number:
20 Addition=30

Explanation

Here, num1.operator +(num2), it indicates that num1 object is calling +() function and num2 object is passed as a parameter, it can also be written as num1+num2;

Program: Return value from operator function:

```

#include<iostream>
using namespace std;
class addition{
public:
    int number,n2,add;
public:

    addition(int x){
        number=x;
    }

    addition(){
        add=0;
    }

    addition operator +(addition x1){
        addition temp;
        temp.add=number + x1.number;
        return temp;
    }
};

int main(){
    addition num1(10);
    addition num2(20);
    addition result;
    result=num1 +num2 ;
    cout<<"\nAddition= "<<result.add;
    return 0;
}

```

Output:

Addition=30

Explanation:

- addition operator +(addition x1),here return type of operator function isaddition.
- temp.add=number + x1.number,here temp is a object of addition typewhich will hold the addition of both the object.
- return temp, here object temp will be returned to calling object.
- Result=num1+num2, here num1 and num2 are the objects of addition type, num1 is calling the +() function and num2 is passed as a parameters,result object will hold values of temp object.

4.3 Operator Overloading using Friend Function:

- If a unary operator is overloaded by friend function, then it will take one argument.
- If a binary operator is overloaded by friend function, then it will take two argument.

Program to overload a unary operator using friend function:

```
#include<iostream>
using namespace std;
class A{
    int a;
    public:
    A(){
        a=10;
    }
    void show(){
        cout<<a;
    }

    friend void operator ++(A&);

};

void operator++(A &obj){
    obj.a=obj.a+1;
}

int main(){
    A obj;
    ++obj;
    obj.show();
    return 0;
}
```

Output:

11

Program: to overload a binary operator using friend function:

```
#include <iostream>
using namespace std;
class Distance {
public:
    int feet, inch;
    Distance(){}
    Distance(int f, int i)
    {
        feet = f;
        inch = i;
    }
    friend Distance operator + (Distance&,Distance&);
};

Distance operator+(Distance& d1,Distance& d2)
{
    Distance d3;
    d3.feet = d1.feet + d2.feet;
    d3.inch = d1.inch + d2.inch;
    return d3;
}

int main()
{
    Distance d1(8, 9);
    Distance d2(10, 2);
    Distance d3;

    d3 = d1 + d2;
    cout << "\nTotal :" << d3.feet <<"Feet " << d3.inch<<"Inches";
    return 0;
}
```

Output:

Total :18Feet 11Inches

Explanation:

- The operator function is implemented outside of the class scope by declaring that function as the friend function.
- In these ways, an operator can be overloaded to perform certain tasks by changing the functionality of operators.

4.4 Restrictions/Rules for operator overloading:

1. Only built-in operators can be overloaded. New operators cannot be created.
2. We cannot overload those operators that are not a part of C++ language.
3. We cannot **overload operators** for built-in data types.
4. Overloaded operators cannot have default argument.
5. There are some operators cannot be overloaded that are given below:
 - ⋮ Scope resolution operator.
 - ⋅ Class membership operator.
 - ?: ternary or conditional operator.
 - sizeof-size of operator
6. When unary operators are overloaded through a member function, take no arguments, but, if they are overloaded by a friend function, takes one argument.
7. When binary operators are overloaded through a member function takes one explicit argument, and if they are overloaded through a friend function takes two explicit arguments.

5 Friend function

- If a function is defined as a friend function in C++, then the protected and private data of a class can be accessed using the function.
- By using the keyword friend compiler knows the given function is a friend function.
- For accessing the data, the declaration of a friend function should be done inside the body of a class starting with the keyword friend.

Declaration of friend function in C++

```
class class_name
{
    friend data_type function_name(argument/s);    // syntax of friend function.
};
```

5.1. Characteristics of a Friend function:

- The function is not in the scope of the class to which it has been declared as a friend.
- It cannot be called using the object as it is not in the scope of that class.
- It can be invoked like a normal function without using the object.
- It cannot access the member names directly and has to use an object name and dot membership operator with the member name.
- It can be declared either in the private or the public part.

Program:

```
#include <iostream>
using namespace std;
class Box{
    private:
        int length;
    public:
        Box(): length(0) { }
        friend int printLength(Box); //friend function
};
int printLength(Box b){
    b.length += 10;
    return b.length;
}
int main(){
    Box b;
    cout<<"Length of box: "<< printLength(b)<<endl;
    return 0;
}
```

Output:

Length of box: 10

5.1. Advantages of Friend Functions

- A friend function is able to access members without the need of inheriting the class.
- The friend function acts as a bridge between two classes by accessing their private data.
- It can be used to increase the versatility of overloaded operators.
- It can be declared either in the public or private or protected part of the class.

5.2. Disadvantages of Friend Functions

- Friend functions have access to private members of a class from outside the class which violates the law of data hiding.
- Friend functions cannot do any run-time polymorphism in their members.

6 C++ Friend class

A friend class can access both private and protected members of the class in which it has been declared as friend.

Program:

```
#include <iostream>
using namespace std;
class A{
    int x =5;
    friend class B;           // friend class.
};

class B{
public:
    void display(A &a){
        cout<<"value of x is : "<<a.x;
    }
};

int main(){
    A a;
    B b;
    b.display(a);
    return 0;
}
```

Output:

value of x is : 5

UNIT-III

1. DYNAMIC OBJECTS

1.1 Pointer to Object

A pointer to a C++ class is done exactly the same way as a pointer to a structure and to access members of a pointer to a class you use the member access operator -> operator, just as you do with pointers to structures. Also as with all pointers, you must initialize the pointer before using it.

Program:

```
#include <iostream>
using namespace std;
class Box {
    public:

    Box(double l = 2.0, double b = 2.0, double h = 2.0) {
        cout << "Constructor called." << endl;
        length = l;
        breadth = b;
        height = h;
    }

    double Volume() {
        return length * breadth * height;
    }

    private:
        double length;    // Length of a box
        double breadth;    // Breadth of a box
        double height;    // Height of a box
};

int main(void) {
    Box Box1(3.3, 1.2, 1.5);
    Box Box2(8.5, 6.0, 2.0);
    Box *ptrBox;
    ptrBox = &Box1;
    cout << "Volume of Box1: " << ptrBox->Volume() << endl;
    ptrBox = &Box2;
    cout << "Volume of Box2: " << ptrBox->Volume() << endl;
    return 0;
}
```

Output:

Constructor called.
Constructor called.
Volume of Box1: 5.94
Volume of Box2: 102

2. Creating And Deleting Dynamic Objects

2.1 New and Delete operators

- Objects, called dynamic objects, may be created whose lifetimes are not bounded by the existence of the scope in which they were created.
- Dynamic object give the programmer greater flexibility in managing objects. However, the programmer also assumes greater responsibility for insuring that dynamic objects are managed properly. Two problems that can occur with dynamic objects are destructing the object "too soon" and failing to destruct the objects at all.
- A dynamic object is created using a "new" operator that returns a pointer to the newly constructed object and is destructed by a "delete" operator.
- A pointer variable is used to hold the pointer to the object that is returned by the "new" operator. The delete operator takes a pointer variable as an operand and destructs the object pointed to by that variable.

Syntax:**Creation of Dynamic Object:**

```
ClassName *ptr_obj;      // pointer to object  
ptr_obj = new ClassName // Dynamic object creation
```

OR

```
ClassName *PtrObj=new ClassName();
```

Deletion of Dynamic Object:

```
delete ptr_obj;          // Delete object dynamically
```


Program to implement the dynamic objects.

```
#include<iostream>
using namespace std;
class Test{
    int a, b;
public:
    Test() {
        cout << "Constructor is called" << endl;
        a = 1;
        b = 2;
    };

    ~Test(){
        cout << "Destructor is called" <<endl;
    }

    void show(){
        cout << "a = " <<a <<endl;
        cout << "b = " <<b <<endl;
    }
};

int main(){
    Test t;           //normal Object
    Test *ptr;        // Dynamic Object Creation Step 1: Declaration
    ptr = new Test;    // Dynamic Object Creation Step 2: Initialization
    Test *Ptr1=new Test(); // (Dynamic object Declation & Initialization in single Line
    ptr->show();
    delete ptr;        // Destroying object dynamically ptr
    cout<<"I am in Main class"<<endl;
    return 0;
}
```

Output:

Constructor is called
Constructor is called
Constructor is called
a = 1
b = 2
Destructor is called
I am in Main class
Destructor is called

2.2 Array Of Object

- An object of a class represents a single record in memory, if we want more than one record of class type, we have to create an array of object. As we know, an array is a collection of similar type, therefore an array can be a collection of class type.
- Arrays in C++ are typically defined using square brackets [] after the type. The index of the array, which ranges from 0 to n - 1, can be used to access each element.

Syntax for Array of object:

```
class-name object_name[ size ];
```

Program to store 3 employees information using array of object.

```
#include<iostream>
using namespace std;
class Employee{
    int Id;
    char Name[25];
    int Age;
    long Salary;
public:

    void GetData() {
        cout<<"\n\tEnter Employee Id : ";
        cin>>Id;
        cout<<"\n\tEnter Employee Name : ";
        cin>>Name;
        cout<<"\n\tEnter Employee Age : ";
        cin>>Age;
        cout<<"\n\tEnter Employee Salary : ";
        cin>>Salary;
    }

    void PutData(){
        cout<<"\n"<<Id<<"\t"<<Name<<"\t"<<Age<<"\t"<<Salary;
    }
};
```

```
int main() {
    Employee E[2];

    for(int i=0;i<2;i++){
        cout<<"\nEnter details of "<<i+1<<" Employee";
        E[i].GetData();
    }
    cout<<"\nDetails of Employees";

    for(int i=0;i<2;i++){
        E[i].PutData();
    }

    return 0;
}
```

Output:

Enter details of 1 Employee
Enter Employee Id : 101
Enter Employee Name : priya
Enter Employee Age : 23
Enter Employee Salary : 10000
Enter details of 2 Employee
Enter Employee Id : 103
Enter Employee Name : soni
Enter Employee Age : 32
Enter Employee Salary : 23000
Details of Employees:
101 priya 23 10000
103 soni 32 23000

2.3 Array of Objects Using Pointers

Array of objects can be defined as an array that's each element is an object of the class. Here we will use the pointer to store the address of an array of objects.

Program:

```
#include<iostream>
using namespace std;
class ShopItem{
    int id;
    float price;
public:

    void setData(int a, float b){
        id = a;
        price = b;
    }
    void getData(void){
        cout<<"Code of this item is "<< id<<endl;
        cout<<"Price of this item is "<<price<<endl;
    }
};

int main(){
    int size;
    ShopItem *ptr = new ShopItem [3];
    ShopItem *ptrTemp = ptr;
    int p, i;
    float q;
    for (i = 0; i < 3; i++){
        cout<<"Enter Id and price of item "<< i+1<<endl;
        cin>>p>>q;
        ptr->setData(p, q); // (*ptr).setData(p, q);
        ptr++;
    }

    for (i = 0; i < 3; i++)
    {
        cout<<"Item number: "<<i+1<<endl;
        ptrTemp->getData();
        ptrTemp++;
    }

    return 0;
}
```

Output:

```
Enter Id and price of item 1
1001
29.5
Enter Id and price of item 2
1002
36.98
Enter Id and price of item 3
1003
52.36
Item number: 1
Code of this item is 1001
Price of this item is 29.5
Item number: 2
Code of this item is 1002
Price of this item is 36.98
Item number: 3
Code of this item is 1003
Price of this item is 52.36
```

2.4 Pointers to Object Members

A variable that holds address of another variable is called pointer. We can declare pointer variables of built-in data types i.e. int, char, float and we can also declare a pointer of user defined data type i.e. Structure similarly we can also declare a pointer object of class type too and this pointer object of class type will hold address of the objects.

A pointer to an object in C++ is used to store the address of an object. For creating a pointer to an object in C++, we use the following syntax:

```
Classname *pointertoobject;
```

For storing the address of an object into a pointer in c++, we use the following syntax:

```
pointertoobject=&objectname;
```

The above syntax can be used to store the address in the pointer to the object. After storing the address in the pointer to the object, the member function can be called using the pointer to the object with the help of an -> **(arrow operator)**.

Program:

```
#include<iostream>
using namespace std;
class Date{
    private:
        short int dd, mm, yy;
    public:
        Date(){
            dd = mm = yy = 0;
        }

        void getdata(int i, int j, int k){
            dd = i;
            mm = j;
            yy = k;
        }

        void prndata(void){
            cout<<"\nData is "<<dd<<"/"<<mm<<"/"<<yy<<"\n";
        }
};

int main(){
    Date D1; //simple object having type Date:
    Date *dptr; //Pointer Object having type Date:
    D1.getdata(19,10,2016);
    cout<<"Printing members using the object ";
    D1.prndata();
    dptr = &D1;
    cout<<"Printing members using the object pointer ";
    dptr->prndata();
    dptr->getdata(20, 10, 2016);
    cout<<"printing members using the object ";
    D1.prndata();
    cout<<"Printing members using the object pointer ";
    dptr->prndata(); //same as (*dptr).prndata();
    return 0;
}
```

Output:

Printing members using the object
Data is 19/10/2016

Printing members using the object pointer
Data is 19/10/2016

printing members using the object
Data is 20/10/2016
Printing members using the object pointer
Data is 20/10/2016

2.5 this Pointer

The **this** pointer in stores the address of the class instance (object), which is called from the member function, to enable functions to access the correct object data members.

Friend functions do not have a **this** pointer, because friends are not members of a class. Only member functions have a **this** pointer.

Following are the situations where 'this' pointer is used:

- a. To Display address of object
- b. When local variable's (parameters) name is same as data member's name.
- c. To Access data members of a class
- d. this operator used in Method Chaining & Return Object

***Use of this operator in Distinguish Data Members**

Program using this pointer to distinguish local members from parameters.

```
#include <iostream>
#include <conio.h>
using namespace std;
class sample{
    int a,b;
    public:
```

```

void input(int a,int b){
    cout<<"Address of object called this function"<<this<<endl;
    this->a=a+b;
    this->b=a-b;
}

void output(){
    cout<<"a = "<<a<<endl<<"b = "<<b;
}

};

int main(){
    sample x;
    x.input(5,8);
    x.output();
    return 0;
}

```

Output:

Address of object called this function0x6ffe10
a = 13
b = -3

***Use of this operator in Method Chaining & Return Object**

program to display the record of student with highest percentage using this pointer.

```

#include<conio.h>
using namespace std;
int count=1;
class student{
    char name[100];
    int age,roll;
    float percent;
public:
    student& number(){
        cout<<"Student : "<<count<<endl;
        count++;
        return *this;
    }
}

```



```
student& getdata(){
    cout<<"Enter data"<<endl;
    cout<<"Name:";
    cin>>name;
    cout<<"Age:";
    cin>>age;
    cout<<"Roll:";
    cin>>roll;
    cout<<"Percent:";
    cin>>percent;
    cout<<endl;
    return *this;
}

student& max(student &s1,student &s2){
    if(percent>s1.percent && percent>s2.percent)
        return *this;
    else if(s1.percent>percent && s1.percent>s2.percent)
        return s1;
    else if(s2.percent>percent && s2.percent>s1.percent)
        return s2;
}

void display(){
    cout<<"Name:"<<name<<endl;
    cout<<"Age:"<<age<<endl;
    cout<<"Roll:"<<roll<<endl;
    cout<<"Percent:"<<percent;
}

};

int main(){
    student s,s1,s2,s3;
    s1.number().getdata();
    s2.number().getdata();
    s3.number().getdata();
    s=s3.max(s1,s2);
    cout<<"Student with highest percentage"<<endl;
    s.display();
    return 0;
}
```

Output:

Student : 1
Enter data
Name:Rahul
Age:18
Roll:1
Percent:45

Student : 2
 Enter data
 Name:Afroz
 Age:19
 Roll:2
 Percent:64

Student : 3
 Enter data
 Name:Aman
 Age:17
 Roll:3
 Percent:76

Student with highest percentage
 Name:Aman
 Age:17
 Roll:3
 Percent:76

3. Inheritance

Inheritance is a mechanism of acquiring the features and behaviors of a class by another class. The class whose members are inherited is called the base class, and the class that inherits those members is called the derived class. Inheritance implements the IS-A relationship.

3.1 Access Control and Inheritance

A table is shown below which shows the behavior of access modifiers when they are derived “public”, “private”, and “protected”.

	Public Derivation	Private Derivation	Protected Derivation
Private members	Not Inherited	Not Inherited	Not Inherited
Protected members	Protected	Private	Protected
Public members	Public	Private	Protected

As shown in Above the table,

PUBLIC MODE

- If the class is inherited in public mode then its private members cannot be inherited in child class.

- If the class is inherited in public mode then its protected members are protected and can be accessed in child class.
- If the class is inherited in public mode then its public members are public and can be accessed inside child class and outside the class.

PRIVATE MODE

- If the class is inherited in private mode then its private members cannot be inherited in child class.
- If the class is inherited in private mode then its protected members are private and cannot be accessed in child class.
- If the class is inherited in private mode then its public members are private and cannot be accessed in child class.

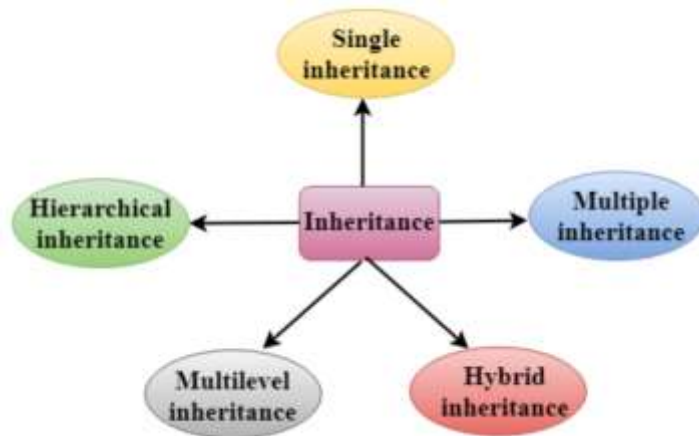
PROTECTED MODE

- If the class is inherited in protected mode then its private members cannot be inherited in child class.
- If the class is inherited in protected mode then its protected members are protected and can be accessed in child class.
- If the class is inherited in protected mode then its public members are protected and can be accessed in child class.

3.2 Different Types of Inheritance

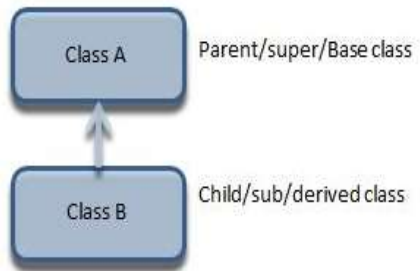
Depending on the way the class is derived or how many base classes a class inherits, we have the following types of inheritance:

- Single Inheritance
- Multiple Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance



1) Single Inheritance

In single inheritance, a class derives from one base class only. This means that there is only one subclass that is derived from one superclass.



Example of Single Inheritance.

```
#include <iostream>
using namespace std;
class Animal{
public:
    int tail=1;
    int legs=4;
};

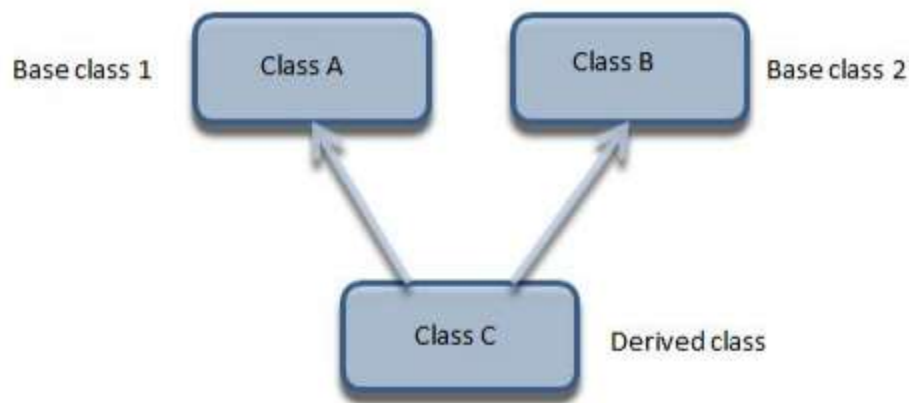
class Dog : public Animal{
public:
    void voiceAction(){
        cout<<"Barks!!!";
    }
};

int main(){
    Dog dog;
    cout<<"Dog has "<<dog.legs<<" legs"<<endl;
    cout<<"Dog has "<<dog.tail<<" tail"<<endl;
    cout<<"Dog ";
    dog.voiceAction();
    return 0;
}
```

Output:

Dog has 4 legs
Dog has 1 tail
Dog Barks!!!

2) Multiple Inheritance



Multiple inheritance is a type of inheritance in which a class derives from more than one classes. As shown in the above diagram, class C is a subclass that has class A and class B as its parent. In a real-life scenario, a child inherits from its father and mother. This can be considered as an example of multiple inheritance.

Example program to demonstrate Multiple Inheritance.

```
#include <iostream>
using namespace std;
class student_marks {
protected:
    int rollNo, marks1, marks2;
public:
    void get() {
        cout << "Enter the Roll No.: ";
        cin >> rollNo;
        cout << "Enter the two highest marks: ";
        cin >> marks1 >> marks2;
    }
};

class cocurricular_marks {
protected:
    int comarks;
public:
    void gets() {
        cout << "Enter the mark for CoCurricular Activities: ";
        cin >> comarks;
    }
};
```

```
class Result : public student_marks, public cocurricular_marks {
    int total_marks, avg_marks;
public:
    void display(){
        total_marks = (marks1 + marks2 + comarks);
        avg_marks = total_marks / 3;
        cout << "\nRoll No: " << rollNo << "\nTotal marks: " <<
            total_marks;
        cout << "\nAverage marks: " << avg_marks;
    }
};

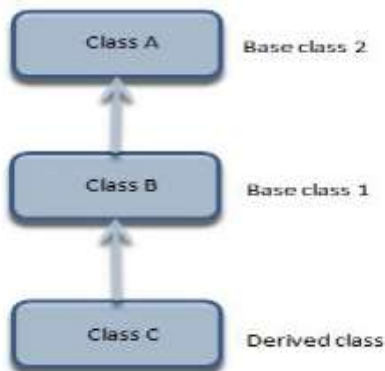
int main(){
    Result res;
    res.get();
    res.getsm();
    res.display();
}
```

Output:

Enter the Roll No.: 25
Enter the two highest marks: 40 50
Enter the mark for CoCurricular Activities: 30
Roll No: 25
Total marks: 120
Average marks: 40

3) Multilevel Inheritance

Multilevel inheritance is represented below.



In multilevel inheritance, a class is derived from another derived class. This inheritance can have as many levels as long as our implementation doesn't go wayward. In the above diagram, class C is derived from Class B. Class B is in turn derived from class A.

Example of Multilevel Inheritance.

```
#include <iostream>
#include <string>
using namespace std;

class Animal {
    string name="";
public:
    int tail=1;
    int legs=4;
};

class Dog : public Animal {
public:
    void voiceAction() {
        cout<<"Barks!!!";
    }
};

class Puppy:public Dog{
public:
    void weeping() {
        cout<<"Weeps!!";
    }
};

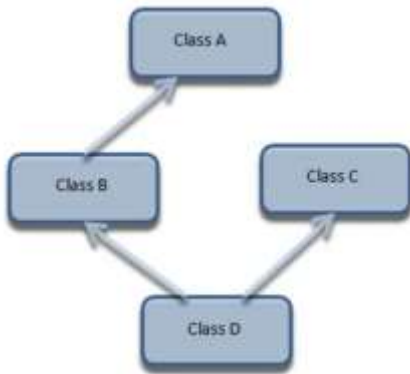
int main() {
    Puppy puppy;
    cout<<"Puppy has "<<puppy.legs<<" legs"<<endl;
    cout<<"Puppy has "<<puppy.tail<<" tail"<<endl;
    cout<<"Puppy ";
    puppy.voiceAction();
    cout<<" Puppy ";
    puppy.weeping();
}
```

Output:

```
Puppy has 4 legs
Puppy has 1 tail
Puppy Barks!!! Puppy Weeps!!
```

4) Hybrid Inheritance

Hybrid inheritance is depicted below.



Hybrid inheritance is usually a combination of more than one type of inheritance. In the above representation, we have multiple inheritance (B, C, and D) and multilevel inheritance (A, B and D) to get a hybrid inheritance.

Example of Hybrid Inheritance.(Hybrid inheritance = multilevel + multiple)

```

#include <iostream>
#include <string>
using namespace std;
class student{                                //First base Class
    int id;
    string name;
    public:
    void getstudent(){
        cout << "Enter student Id and student name"; cin >> id >>
name;
    }
};

class marks: public student{                  //derived from student
    protected:
        int marks_math,marks_phy,marks_chem;
    public:
        void getmarks(){
            cout << "Enter 3 subject marks:";
            cin >>marks_math>>marks_phy>>marks_chem;
        }
};
  
```



```
class sports{
    protected:
    int spmarks;
    public:
    void getsports(){
        cout << "Enter sports marks:";
        cin >> spmarks;
    }
};

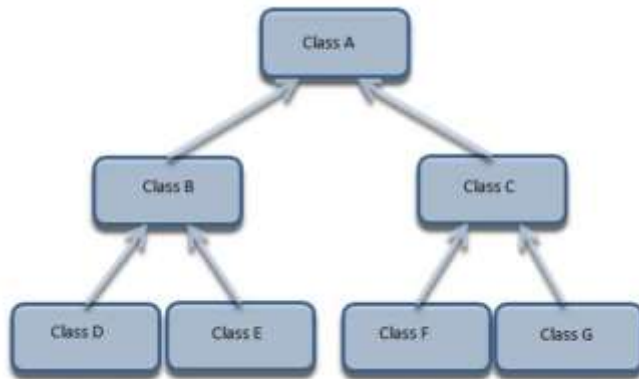
class result:public marks, public sports{ //Derived class by multiple inheritance
    int total_marks;
    float avg_marks;
    public :
    void display(){
        total_marks=marks_math+marks_phy+marks_chem;
        avg_marks=total_marks/3.0;
        cout << "Total marks =" << total_marks << endl;
        cout << "Average marks =" << avg_marks << endl;
        cout << "Average + Sports marks =" << avg_marks+spmarks;
    }
};

int main(){
    result res; //object//
    res.getstudent();
    res.getmarks();
    res.getsports();
    res.display();
    return 0;
}
```

Output:

```
Enter student Id and student name 25 Ved
Enter 3 subject marks:89 88 87
Enter sports marks:40
Total marks =264
Average marks =88
Average + Sports marks =128
```

5) Hierarchical Inheritance



In hierarchical inheritance, more than one class inherits from a single base class as shown in the representation above. This gives it a structure of a hierarchy.

Example demonstrating Hierarchical Inheritance.

```
#include <iostream>
using namespace std;
class Shape {
public:
int x,y;
void get_data(int n,int m) {
    x= n;
    y = m;
}
};

class Rectangle : public Shape {
public:
int area_rect() {
    int area = x*y;
    return area;
}
};

class Triangle : public Shape {
public:
int triangle_area() {
    float area = 0.5*x*y;
    return area;
}
};
```

```
class Square : public Shape {
public:
    int square_area() {
        float area = 4*x;
        return area;
    }
};

int main() {
    Rectangle r;
    Triangle t;
    Square s;
    int length,breadth,base,height,side;
    cout << "Enter the length and breadth of a rectangle: ";
    cin>>length>>breadth;

    r.get_data(length,breadth);
    int rect_area = r.area_rect();

    cout << "Area of the rectangle = " <<rect_area<<endl;
    cout << "Enter the base and height of the triangle: ";
    cin>>base>>height;
    t.get_data(base,height);

    float tri_area = t.triangle_area();
    cout <<"Area of the triangle = " << tri_area<<endl;
    cout << "Enter the length of one side of the square: ";
    cin>>side;

    s.get_data(side,side);
    int sq_area = s.square_area();
    cout <<"Area of the square = " << sq_area<<endl;
    return 0;
}
```

Output:

```
Enter the length and breadth of a rectangle: 10 5
Area of the rectangle = 50
Enter the base and height of the triangle: 4 8
Area of the triangle = 16
Enter the length of one side of the square: 5
Area of the square = 20
```

3.3 What is the Diamond Problem?

- When we inherit more than one base class in the same derived class and all these base classes also inherit another but same single class (super parent), multiple references of the super parent class become available to the derived class.
- So, it becomes unclear to the derived class, which version of the super parent class it should refer to.

Consider the following program for instance:

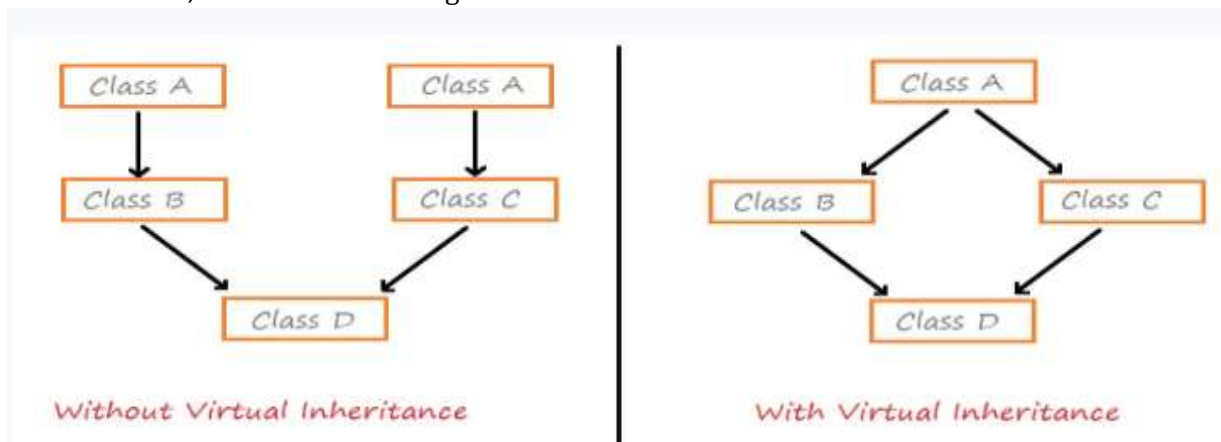
```
#include <iostream>
using namespace std;
class A{ //super parent class
public:
    void name(){
        cout << "This is class A \n";
    }
};
class B: public A{}; //base class I
class C: public A{}; //base class II
class D: public B, public C{}; //derived class

int main(){
    D d;
    d.name();
    return 0;
}
```

Output:

line 24|error: request for member 'name' is ambiguous

- There is no syntactical error in the above program but still, if we try to compile then it returns the following compilation error:
- This is because two instances of class A's name() method is available for class D, one through class B, and the other through class C.



- In this case, the compiler gets confused and cannot decide which name() method it should refer to.

- This ambiguity often occurs in the case of multiple inheritances and is popularly known as the **diamond problem** in C++.
- To remove this ambiguity, we use virtual inheritance to inherit the super parent.

3.3.1 What is Virtual Inheritance?

- Virtual inheritance in C++ is a type of inheritance that ensures that only one copy or instance of the base class's members is inherited by the grandchild derived class.
- It is implemented by prefixing the virtual keyword in the inheritance statement.

Example:

```
class A: virtual public B{  
    //members  
}
```

- If we now apply virtual inheritance in our previous example, only one instance of class A would be inherited by class D (i.e. $B::A$ and $C::A$ will be treated as the same).

Program:

```
#include <iostream>  
using namespace std;  
class A{  
public:  
    void name(){  
        cout << "This is class A \n";  
    }  
};  
  
class B: virtual public A{};  
class C: virtual public A{};  
class D: public B, public C{};  
  
int main(){  
    D d;  
    d.name();  
    return 0;  
}
```

Output:

This is Class A

- Now because there is only one reference of class A's instance is available to the child classes, we get the proper diamond-shaped inheritance.

- Also, If we need to override any of the super parent class's method (class A) in the child classes, we should override it till the very derived class, like the following. Otherwise, the compiler will throw **no unique overrider error**.

Program:

```
#include <iostream>
using namespace std;

class A{
public:
    virtual void name(){
        cout << "This is class A \n";
    }
};

class B: virtual public A{
public:
    void name(){
        cout << "This is class B \n";
    }
};

class C: virtual public A{
public:
    void name(){
        cout << "This is class C \n";
    }
};

class D: public B, public C{
public:
    void name(){
        cout << "This is class D \n";
    }
};

int main(){
    D d;
    d.name();
    return 0;
}
```

Output:

This is class D

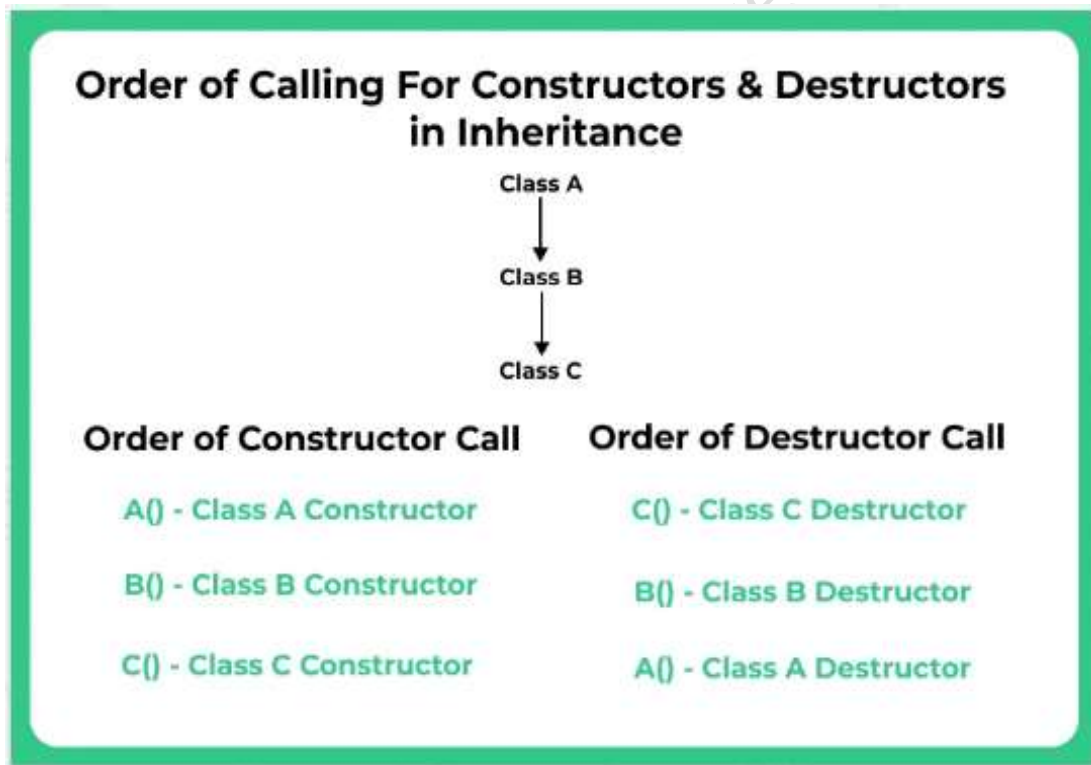
4. Constructor and Destructor in Derived classes

4.1 Constructors in inheritance:

- Base class constructors are always called in the derived class constructors. Whenever you create derived class object, first the base class default constructor is executed and then the derived class's constructor finishes execution.

4.2 Destructors in inheritance:

- Like constructor, destructor is also a member function of a class that has the same name as the class name preceded by a tilde(~) operator. It helps to deallocate the memory of an object.
- Destructor is called in reverse sequence of constructor invocation i.e. The destructor of the derived class is called first and the destructor of the base is called next.
- Destructors are called automatically at the end of the program.



C++ program to the sequence of execution of constructor and destructor inheritance Program:

```
#include <iostream>
using namespace std;

class parent{           //parent class
public:

    parent(){           //constructor
        cout << "Parent class Constructor\n";
    }

    ~parent(){          //destructor
        cout << "Parent class Destructor\n";
    }
};

class child : public parent{   //child class
public:

    child(){             //constructor
        cout << "Child class Constructor\n";
    }

    ~ child(){            //destructor
        cout << "Child class Destructor\n";
    }
};

int main(){
    child c;
    return 0;
}
```

Output:

```
Parent class Constructor
Child class Constructor
Child class Destructor
Parent class Destructor
```


UNIT-IV

1. Importance/Need of virtual function

We use virtual functions to ensure that the correct function is called for an object, regardless of the reference type used to call the function. They are basically used to achieve the runtime polymorphism and are declared in the base class by using the virtual keyword before the function.

2. What is Virtual Function?

- Virtual function is a member function in the base class that you redefine in a derived class. It is declared using the virtual keyword.
- It is used to tell the compiler to perform dynamic binding or late binding on the function.
- When the function is made virtual, C++ determines which function is to be invoked at the runtime based on the type of the object pointed by the base class pointer.
- If you define a virtual function in the base class then it is not necessary to re-define again in derived class. In this case call will invokes the base-virtual function.

Defining Virtual Function Syntax:

```
class class_name{
Public:
virtual return_type function_name(){
-----
----- // body of virtual function
-----
}
};
```

Example for Virtual Function Program

```
#include<iostream>
using namespace std;

class base{
public:

void function_1() {
cout << "base class function 1\n";
}

virtual void function_2(){
cout << "base class function 2\n";
}

virtual void function_3(){
cout << "base class function 3\n";
}

virtual void function_4(){
cout << "base class function 4\n";
}
};

class derived : public base{
public:

void function_1(){
cout << "derived class function 1\n";
}

void function_2(){
cout << "derived class function 2\n";
}

void function_4(int x) {
cout << "derived class function 4\n";
}
};

int main() {
base *ptr;
derived obj1;
ptr = &obj1;
ptr->function_1();
ptr->function_2();
ptr->function_3();
```

```
ptr->function_4();  
    return 0;  
}
```

Output-

base class function 1
derived class function 2
base class function 3
base class function 4

Explanation

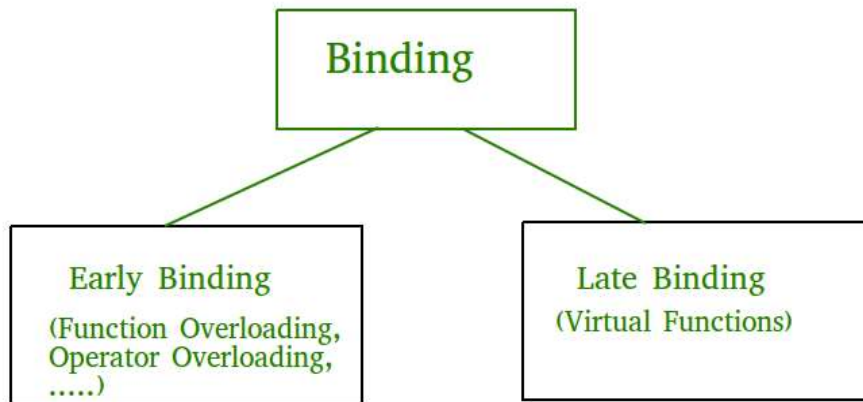
For function_1() function call, base class version of function is called, function_2() is overridden in derived class so derived class version is called, function_3() is not overridden in derived class and is virtual function so base class version is called, similarly function_4() is not overridden so base class version is called.

2.1. Rules for Virtual Function in C++

1. Virtual functions must be members of some class.
2. The prototype of these functions should be the same for both the base and derived class
3. Derive them using the "virtual" keyword
4. Virtual functions cannot be static members.
5. They are accessed through object pointers.
6. They can be a friend of another class.
7. The function call resolving is done at run-time
8. Redefining the virtual function in the derived class is optional, but it needs to be defined in the base class
9. You can create a virtual destructor but not a constructor

3. What is binding?

Binding refers to the process of converting identifiers (such as variable and performance names) into addresses. Binding is done for each variable and functions. For functions, it means that matching the call with the right function definition by the compiler. It takes place either at compile time or at runtime.



3.1. Early Binding (compile-time time polymorphism)

As the name indicates, compiler (or linker) directly associate an address to the function call. It replaces the call with a machine language instruction that tells the mainframe to leap to the address of the function.

Example Program:

```
#include <iostream>
Using namespace std;
class Animals{
public:

void sound(){
cout << "Genric animal sound" << endl;
}
};

class Cats: public Animals{
public:
void sound(){
cout << "Cat meow" << endl;
}
};

int main(){
Animals *a;
Cats c;
a= &c;
a -> sound(); // early binding
return 0;
}
```

Output: Genric animal sound

Explanation

In this example, we created a pointer `a` to the parent class `Animals`. Then by writing `a = &c`, the pointer '`a`' started referring to the object `c` of the class `Cats`.
`a -> sound()`; – On calling the function `sound()` which is present in both the classes by the pointer '`a`', the function of the parent class got called, even if the pointer is referring to the object of the class `Cats`.

3.2. Late Binding : (Run time polymorphism)

In this, the compiler adds code that identifies the kind of object at runtime then matches the call with the right function definition. This can be achieved by declaring a **virtual function**.

Example Program:

```
#include<iostream>
using namespace std;
class Animals{
public:
virtual void sound(){
cout << "Genric animal sound" << endl;
}
};

class Cats: public Animals{
public:
void sound(){
cout << "Cats meow" << endl;
}
};

int main(){
Animals *a;
Cats c;
a = &c;
a -> sound();
}
```

Output:

Cats meow

Explanation

Here the function `sound()` of the base class is made virtual, thus the compiler now performs latebinding for this function. Now, the function call of the `sound()` function will be matched to the function definition at runtime. Since the compiler now identifies pointer 'a' as referring to the object 'c' of the derived class Cats, it will call the `sound()` function of the class Cats.

3.3. Difference between Early binding and Late binding:

Early Binding	Late binding
✚ Early binding is also known as static binding.	✚ Late binding is also known as dynamic binding.
✚ It happens at compile time.	✚ It happens at run time.
✚ Uses function overloading and operator overloading.	✚ Uses function overriding and virtual function.
✚ This is called compile time polymorphism.	✚ This is called run time polymorphism.
✚ Type of object is determined at compile time.	✚ Type of object is determined at run time.
✚ Eg: <code>Derived d;</code>	✚ Eg: <code>Base *p= new Derived();</code>

4. Pure Virtual Function

- Sometimes Definition(implementation) of all function cannot be provided in a base class because we don't know the implementation. Such class is called abstract class. When the function has no definition, such function is known as "**do-nothing**" function.
- The "**do-nothing**" function is known as a **pure virtual function**. A pure virtual function is a function declared in the base class that has no definition relative to the base class.
- A class containing the pure virtual function cannot be used to declare the objects of its own, such classes are known as **abstract base classes**.
- The main objective of the base class is to provide the traits to the derived classes and to create the base pointer used for achieving the runtime polymorphism.

4.1. Declaration Of Pure Virtual Function:

A pure virtual function (or abstract function) in C++ is a virtual function for which we don't have implementation, we only declare it. A pure virtual function is declared by assigning 0(ZERO) in declaration.

Example:

```
class Test {  
public:  
    virtual void show() = 0;  
};
```

Here, show() is pure virtual function which has no definition, and Test is an Abstract class because it has pure virtual function and we can not create object of Test class.

5. Abstract Class

- An abstract class in C++ has at least one pure virtual function by definition.
- In other words, a function that has no definition and these classes cannot be instantiated.
- The abstract class's child classes must provide body to the pure virtual function; otherwise, the child class would become an abstract class in its own right.

5.1. Key Points of Abstract class in C++:

1. The class having a pure virtual function is an abstract class
2. An abstract class can have concrete also.
3. The object of abstract class cannot be created
4. A derived class must override pure virtual function, otherwise, it will also become an abstract class.
5. The pointer of the abstract class can be created
6. The pointer of the abstract class can hold the object of the derived class
7. Abstract classes are used for achieving polymorphism
8. The base class can be concrete
9. Abstract with some concrete and some pure virtual functions

Example program:

```
#include <iostream>
using namespace std;

class Employee{
    virtual int getSalary() = 0;           // abstract base class
                                           // pure virtual function
};

class Employee_1: public Employee{
    int salary;
    Public:
    Employee_1(int s){
        salary = s;
    }

    int getSalary(){
        return salary;
    }
};

class Employee_2: public Employee{
    int salary;
    Public:

    Employee_2(int t){
        salary = t;
    }

    int getSalary(){
        return salary;
    }
};

int main(){
    Employee_1 e1(5000);
    Employee_2 e2(3000);
    int a, b;
    a = e1.getSalary();
    b = e2.getSalary();
    cout << "Salary of Developer : " << a << endl;
    cout << "Salary of Driver : " << b << endl;
    return 0;
}
```

Output:

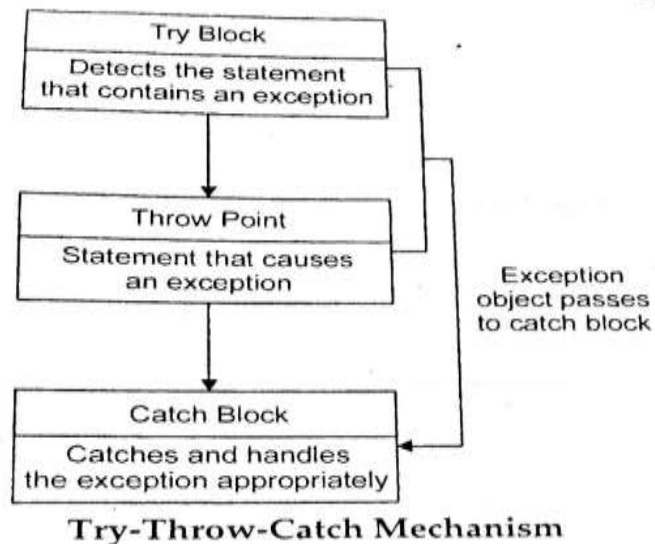
Salary of Developer:5000
Salary of Driver:3000

Explanation

The 'getSalary()' function in the class Employee is a pure virtual function. Since the Employee class contains the pure virtual function, therefore it is an abstract base class. Since the pure virtual function is defined in the subclasses, therefore the function 'getSalary()' is defined in both the subclasses of the class Employee i.e in Employee_1 and Employee_2.

6. Exception Handling Model

In C++, exception handling is a mechanism to handle errors and exceptional situations that occur during the execution of a program. It allows a program to transfer control to a designated handler when an exceptional situation arises.



The C++ exception handling model is based on three keywords: **try**, **throw**, and **catch**.

1. **try** block: The **try** block contains the code that may throw an exception. If an exception is thrown within the **try** block, the program will terminate normal execution and jump to the appropriate **catch** block.
2. **throw** statement: The **throw** statement is used to signal an exception. When an exception is thrown, the program will immediately transfer control to the first matching **catch** block.
3. **catch** block: The **catch** block is used to handle the thrown exception. It contains the code that will handle the exception and bring the program back to normal execution.

The basic syntax for exception handling in C++ is as follows:

```
try {                                // Block of code to try
    throw exception;                // Throw an exception when a problem arise
}
catch (exception_type1 ex1) {}      // Handling code for exception_type1
catch (exception_type2 ex2) {}      // Handling code for exception_type2
catch (...) {}                      // Handling code for any other exception
```

- Here, **exception_Type** is the type of the exception that you want to catch. You can catch specific exceptions by specifying their type, or you can catch all exceptions by using the catch-all ... syntax.
- **try** encloses the statements that may throw an exception. The **catch** blocks handle the thrown exceptions, where **exception_type1**, **exception_type2**, and so on are the types of exceptions that can be caught. The ellipsis (...) represents a catch block that catches any other exception that is not caught by the preceding catch blocks.
- When an exception is thrown, the program checks each catch block in the order in which they appear in the code until it finds a catch block that matches the type of the thrown exception. If no catch block matches the exception, the program terminates abnormally.
- In addition to the basic **try-catch** model, C++ also provides the **throw()** specification to indicate that a function can throw an exception, and the **noexcept** specification to indicate that a function will not throw an exception.

Example Program to check the age to Access the content:

```
#include <iostream>
using namespace std;
int main() {
    try {
        int age;
        cout<<"enter Your Age";
        cin>>age;

        if (age >= 18) {
            cout << "Access granted - you are old enough.";
        }

        else {
            throw (age);
        }

    }

    catch (int myNum) {
        cout << "Access denied - You must be at least 18 years old.\n";
        cout << "Age is: " << myNum;
    }

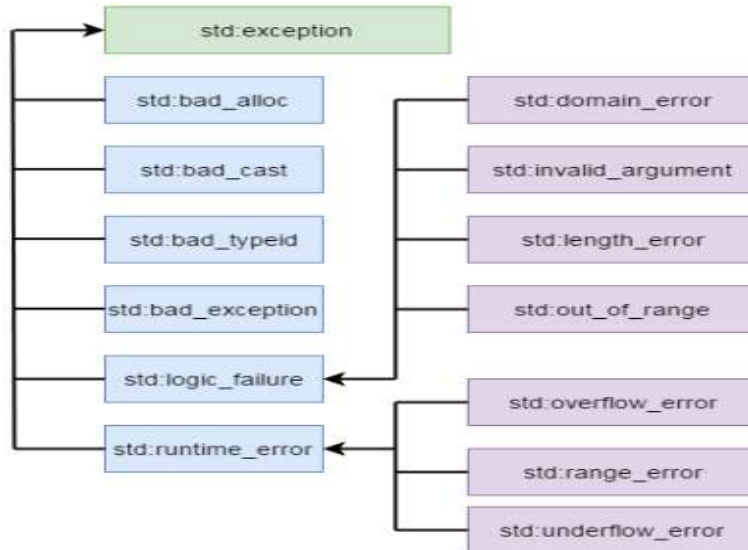
    return 0;
}
```

Output:

```
Enter Your Age
17
Access denied - You must be at least 18 years old.
Age is: 17
```

7. List of Exceptions:

C++ provides several built-in exception classes that can be used to handle different types of errors and unexpected situations that may occur during the execution of a program. Here are some of the common exception classes available in C++:



Exception	Description
std::exception	Exception and parent class of all standard C++ exceptions.
std::bad_alloc	Generally thrown by new.
std::bad_cast	Generally thrown by dynamic_cast.
std::bad_typeid	Generally thrown by typeid.
std::bad_exception	Thrown when an unexpected exception occurs during exception handling.
std::logic_failure	Can be detected by reading code.
std::runtime_error	Cannot be detected by reading code.

std::domain_error	Thrown when using a mathematically invalid domain.
std::invalid_argument	Thrown when using invalid arguments.
std::length_error	Thrown when a large std::string is created.
std::out_of_range	Thrown by the at method.
std::overflow_error	Thrown when a mathematical overflow occurs.
std::range_error	Thrown when attempting to store an out-of-range value.
std::underflow_error	Thrown when a mathematical underflow occurs.

- In addition to these exceptions, C++ also provides several other exception classes for specific use cases. Programmers can also create their own custom exception classes by deriving from the **std::exception** class or any of its subclasses.

Steps to create user define Exceptions

1 Inherit the exception class

2 override the what() method/function of Exception class

Syntax to override what() method

```
virtual const char* what() const throw(){
return "Message"
}
```

Explanation of Syntax

virtual=> the function may be overridden in the derived class

const char*=> this returns modifiable pointer to constant array of character

what()=>print the error message

const=>the function does not modifying the members of object on which it invoked

throw()=>Exception specification , it represent what() is not going to throw any kind of exception

Example Program to check divide by Zero Exception using user define Exceptions

```

#include<iostream>
using namespace std;
class myexception: public exception{                                //step 1

virtual const char* what() const throw(){                            //step 2

    return "Divide by Zero";

}

};

int main(){
    int a,b;
    cout<<"Enter two number to get it's division\n";
    cin>>a>>b;

    try{
        if(b==0){
            myexception e;
            throw e;
        }
        else{
            cout<<"The division operation is : "<<a/b;
        }
    }
    catch(exception &m){
        cout<<"The exception caught here : " <<m.what();
    }
}

```

Output:

```

Enter two number to get it's division
78
0
The exception caught here : Divide by Zero

```

8. Customizing termination behavior for uncaught exception

- If none of the catch handlers for a try block matches a thrown exception the exception moves to the next enclosing try block.
- If there is no match in any enclosing try block the exception is uncaught. An uncaught exception also occurs if a new exception is thrown before an existing one is handled. Cleanups may fail to occur with an uncaught exception, so this is an error.
- The exception handling subsystem calls the function: **unexpected()**. This function, provided by the default C++ library, defines the behavior when an uncaught exception

arises. By default, unexpected calls **terminate()**. The process is aborted.

- The **terminate()** and **unexpected()** simply call other functions to actually handle an error. That is **terminate()** calls **abort()**, and **unexpected()** calls **terminate()**. Thus, both functions halt the program execution when an exception handling error occurs. i.e [**unexpected() => terminate() => abort()**]
- However, you can change the way termination occurs. To change the terminate handler, the function used is **set_terminate(terminate_handler newhandler)**, which is defined in the header **<exception>**.

The following program demonstrates how to set a custom termination handler:

CPP program to set a new termination handler for uncaught exceptions.

```
#include <exception>
#include <iostream>
using namespace std;
void myhandler()
{
    cout << "Terminating program due to uncaught exception\n";
    abort();
}
int main(){
    int p;
    cout<<"Enter Your Percentage\n";
    cin>>p;

    set_terminate(myhandler);           // set new terminate handler

    try {
        if(p<0 || p>100){
            throw p;
        }
        else{
            cout<<"\n Your Percentage is :"<<p;
        }
    }
    catch (char a) {                    // won't catch an int exception
        cout << "Exception found\n";
    }
    return 0;
}
```

Output:

```
Enter Your Percentage 100.36
Terminating program due to uncaught exception
```

9. Fault Tolerant Design Techniques

Fault tolerance is the ability of a system to continue operating in the presence of faults or errors. In C++, there are several techniques you can use to design fault-tolerant systems. Here are some common techniques:

1. **Exception Handling:** C++ provides a built-in exception handling mechanism to handle runtime errors that may occur during program execution. By handling exceptions properly, you can ensure that your program continues to function correctly in the presence of errors.
2. **Input Validation:** Input validation is a technique that checks the input data to ensure that it meets the expected format and range of values. By validating input data, you can prevent errors and faults that may occur due to incorrect input.
3. **Defensive Programming:** Defensive programming is a technique that involves writing code that can handle unexpected situations or errors. This involves adding error checking and handling code to functions and methods to ensure that they can continue to operate in the presence of errors.
4. **Design for Failure:** Designing for failure involves building systems that can detect and recover from faults or errors. This can be achieved by using redundancy, monitoring, and self-healing techniques. For example, redundant components can be used to ensure that the system can continue operating even if one component fails.
5. **Testing:** Testing is a critical component of building fault-tolerant systems. By testing the system under different conditions and scenarios, you can identify and fix errors and faults before they occur in production.
6. **Logging and Debugging:** Logging and debugging techniques can be used to identify errors and faults that occur during the execution of the program. By logging events and error messages, you can identify patterns and trends that may indicate potential faults or errors.
7. **Recovery Mechanisms:** Recovery mechanisms are techniques that are used to recover from faults or errors. This may involve retrying failed operations, rolling back transactions, or using alternative paths to achieve the desired outcome.

Overall, building fault-tolerant systems requires a combination of good programming practices, error handling techniques, and design strategies. By adopting these techniques, you can build systems that are more resilient and can continue to operate in the presence of faults or errors.

10. Memory Allocation Failure Exception

The **bad_alloc** exception is thrown by the C++ standard library when the program fails to allocate memory dynamically using **new**. Here are some techniques to handle this exception in a C++ program:

1. **Catch the exception:** One way to handle the **bad_alloc** exception is to catch it using a **try-catch** block. When the **new** operator fails to allocate memory, it throws the **bad_alloc** exception. By wrapping the **new** operation in a **try** block and catching the **bad_alloc** exception, you can gracefully handle the error and take appropriate action.

Program:

```
#include <iostream>
using namespace std;
int main ()
{
    try {
        int* p = new int[1000000000000000000000000000000];
    }
    catch (const bad_alloc& e) {
        cout<< "Error: " << e.what();    // handle the exception
    }
    return 0;
}
```

Output:

Error: std::bad_alloc

2. Use **nothrow**: The **nothrow** parameter can be passed to the **new** operator to prevent it from throwing an exception when it fails to allocate memory. Instead, it returns a null pointer. You can check for the null pointer and handle the error accordingly.

Program:

```
#include <iostream>
using namespace std;
int main ()
{
    int* p = new (nothrow) int[1000000000000000000000000000000];
    if (p == NULL) {
        cout<< "Error: Out of memory";    // handle the error
    }

    return 0;
}
```

Output:

Error: Out of memory

3. **Reduce memory usage**: If your program frequently runs out of memory, consider reducing its memory usage. This can be achieved by optimizing your algorithms and data structures, freeing memory that is no longer needed, or using more efficient memory management techniques.

Overall, handling the `bad_alloc` exception requires a combination of good programming practices, error handling techniques, and memory management strategies.

11. Rules for Handling Exception Successfully

1. **Only handle exceptions that you can handle:** Catch only those exceptions that you can handle. For example, if you catch a `std::bad_alloc` exception, you should handle it by freeing up memory or reporting an error, rather than simply re-throwing the exception.
2. **Catch exceptions by reference:** Catch exceptions by reference rather than by value. This avoids unnecessary copying of the exception object and ensures that the original exception object is preserved.

```
try {
    // some code that may throw an exception
}
catch (const std::exception& e) {} // handle the exception
```

3. **Use the appropriate catch block:** Catch specific exceptions using the appropriate catch block. For example, catch `std::logic_error` for logical errors, `std::runtime_error` for runtime errors, and `std::exception` for general exceptions.

```
try {
    // some code that may throw an exception
}
catch (const std::logic_error& e) {} // handle a logical error
catch (const std::runtime_error& e) {} // handle a runtime error
catch (const std::exception& e) {} // handle any other exception
```

4. **Use RAII:** Use RAII (Resource Acquisition Is Initialization) to manage resources such as memory, file handles, and network connections. This ensures that resources are properly cleaned up when an exception is thrown, even if the code that threw the exception didn't explicitly free the resources.
5. **Don't catch exceptions in a loop:** Don't catch exceptions in a loop unless you're sure that the loop will eventually terminate. Catching exceptions in a loop can lead to an infinite loop if the exception is not handled properly.
6. **Don't ignore exceptions:** Don't ignore exceptions. Always handle them or re-throw them if you can't handle them.
7. **Be consistent:** Be consistent in your exception handling. Use the same exception handling style throughout your codebase to make it easier to read and maintain.